



MUKESH PATEL SCHOOL OF
TECHNOLOGY MANAGEMENT
& ENGINEERINGTM

Mukesh Patel School of Technology and Management, Mumbai

MBA Tech IT



SAPHIRE
International Airport

Software Engineering Project · October 2026

Saphire AOCS

Airport Operations Coordination System

A full-stack platform that coordinates flight turnaround, gate assignment, check-in, baggage reconciliation, runway status and airline billing for one hub airport.

43

Relational tables

158k+

Seed records

16

Pages and consoles

109

Automated tests

Submitted for evaluation · 5 October 2026

Meet the Project Team

Four members, each owning one layer of Sapphire AOCS, from the database to the interface



Krishna Solanki

I075

Database & System Integration
Lead

PostgreSQL schema, Flyway
migrations, seed data and API
integration



Chaitanya Tikku

I078

Documentation, UML & QA Lead

SRS, all UML and database
diagrams, and the automated test
suite



Anuvrat Tripathi

I080

Frontend UI/UX Lead

React 19 and TypeScript interface,
Material UI design system, public
portal



Anay Modi

I088

Backend API & Logic Lead

Spring Boot 3 services, JWT
security, gate and turnaround
dispatch logic

Our Project Mentor

Saphire AOCS was developed under the guidance of our faculty mentor



Project mentor

Dr. Dharmesh Rathod

Assistant Professor

NMIMS Mukesh Patel School of Technology Management & Engineering, Mumbai



Mentor of the Saphire AOCS project

The Sapphire International Masterplan

Modelled on CSMIA Mumbai: an X-shaped terminal with radial concourse limbs and parallel runways



Terminal Topography

X-shaped central terminal with radial concourse limbs (Concourses A, B and C in the live data).



Gates & Stands

248 gates and 248 stands (71 remote), with boarding bridges, ground power and pre-conditioned air.



Parallel Runway System

Independent IFR arrivals and departures; each runway carries a live status such as CAT III.



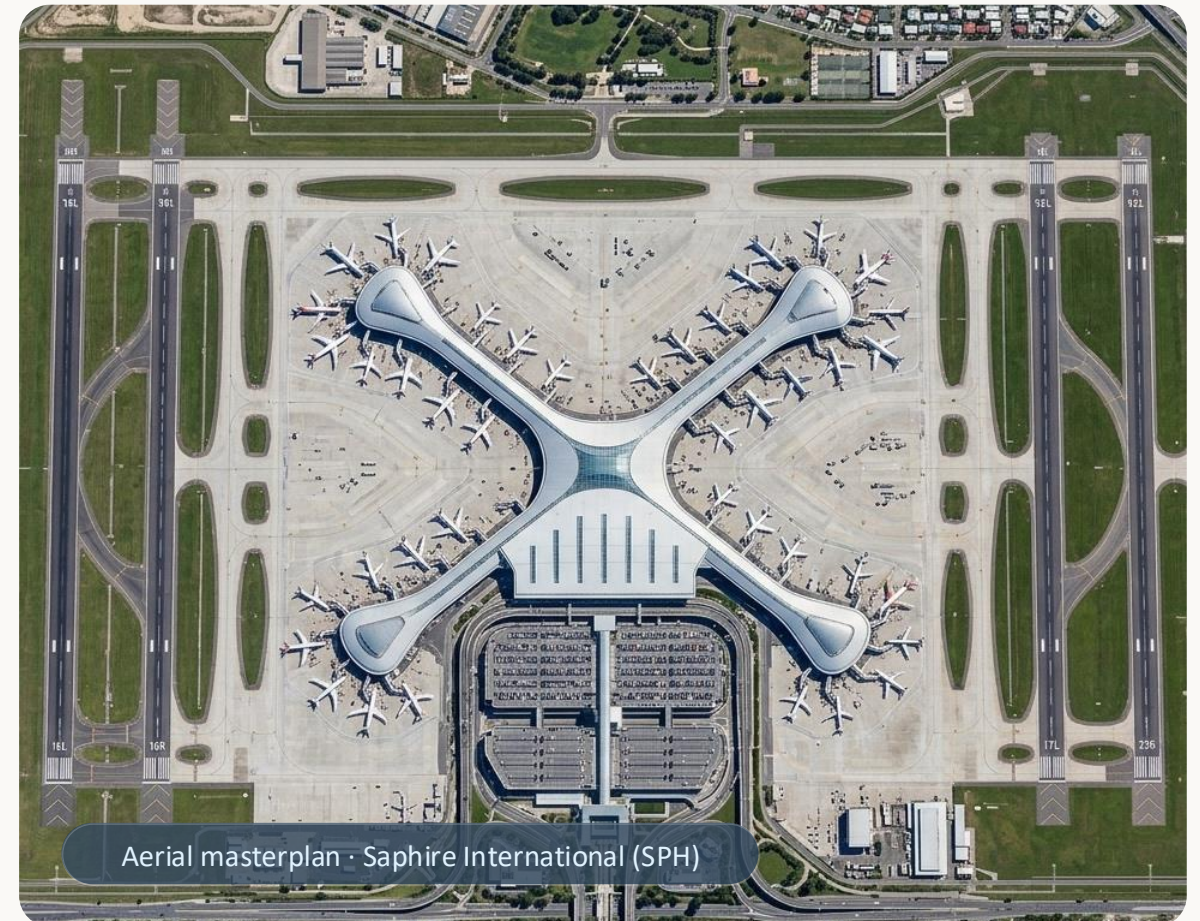
Baggage Transit Corridors

Underground conveyors and tilt-tray sorters link check-in desks to aircraft stands.



Logistics & Cargo City

Multi-bay freight terminal with HAZMAT vaults and customs staging.



Problem Definition & Operational Scope

Six pain points across traditional airport operations, and the platform that answers them



Operational Silos

Airlines, ground handlers, security and ATC run on separate systems, so updates arrive late and delays cascade.



Turnaround Bottlenecks

Fuelling, catering, cleaning and baggage work have no shared schedule or real-time delay attribution.



Gate Allocation Conflicts

Gates are assigned by hand without wingspan checks, risking unsafe stand use and clashes.



Baggage Tracing Gaps

No end-to-end bag tag tracking from screening and sorting to the apron and reclaim belt.



Tariff Billing Leakage

Landing, parking and aerobridge fees are consolidated by hand, which leaks revenue and delays invoices.



Solution: Sapphire AOCS

One real-time A-CDM platform linking check-in, turnaround, security clearance and billing on a single relational core.

User Stories & Acceptance Criteria

Four primary personas, each story paired with testable acceptance criteria



AOCC

AOCC Duty Controller

“As an AOCC duty manager, I want live turnaround progress for every active flight, so I can spot delay risks early and log IATA delay codes.”

Acceptance criteria

- ✓ Progress per active flight
- ✓ IATA delay codes on delays
- ✓ Delay log on the AOCC board



DCS

Check-In Agent

“As a check-in agent, I want to find a booking by PNR, choose a seat on the cabin map and tag bags, so I can issue a boarding pass.”

Acceptance criteria

- ✓ Lookup by PNR or passport
- ✓ Seat clash returns 409
- ✓ BCBP barcode and bag tag



Airside

Airside Dispatcher

“As an airside officer, I want gate checks against aircraft wingspan, so a widebody is never parked at a gate that is too narrow.”

Acceptance criteria

- ✓ Wingspan vs gate limit
- ✓ Conflicts rejected with 409
- ✓ Live gate occupancy board



Public

Passenger

“As a passenger, I want live flight status, baggage tracking and online lost-and-found reports without needing an account.”

Acceptance criteria

- ✓ No login for public pages
- ✓ Tracked inquiry tickets
- ✓ Claimant contacts private

Non-Functional Requirements

Targets for performance, availability, security and data integrity



Performance & Latency

350 ms

- REST responses within 350 ms at 1,000 concurrent requests
- Every paged endpoint capped at 100 rows
- Indexes on foreign keys and common filter columns



Availability & Scalability

99.9%

- Stateless Spring Boot services behind Nginx
- Multi-stage Docker images and Docker Compose
- Health endpoint for container checks



Security & Sessions

HS256

- Signed JWT carries a session ID checked on every request
- One active session per user, revocable server-side
- BCrypt password hashes, never serialised



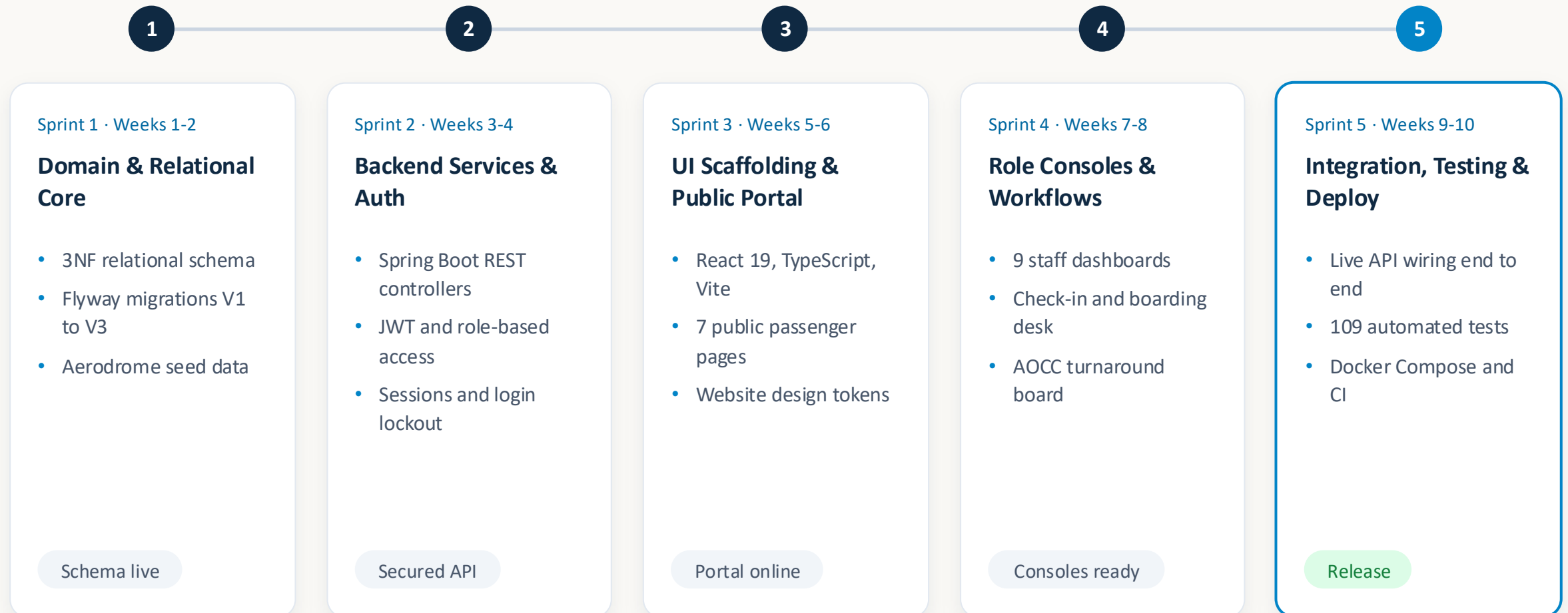
Data Integrity

3NF

- 43 tables in third normal form
- ON DELETE RESTRICT on clearance and audit records
- Unique seat per flight; wingspan checked per gate

Agile Scrum Process & Sprint Cadence

Five two-week sprints, from domain modelling to an integrated, tested release



UML Use Case Model

One system boundary, eight actors and the include and extend relationships between use cases



Actors

8 actors · 10 RBAC roles in the database

- Passenger
- Check-in agent
- Security officer
- Baggage handler
- AOCC manager
- Airside dispatcher
- Ground supervisor
- Billing and admin

Relationships

- «include» Process Pax Check-In → Select Seat Map
- «include» Process Pax Check-In → Issue IATA Pass & Tag
- «extend» Detect Wingspan Conflict → Assign Gate & Stand
- «extend» Log IATA Delay Code → Dispatch Turnaround

Use Case Specifications

Actors, preconditions, main flow, postconditions and the exception path for two core operations

UC-05

DCS Check-In

Actor	Check-in agent (CHECKIN_AGENT role)
Preconditions	Flight is SCHEDULED or BOARDING; the counter is assigned
Main flow	1 Agent searches by PNR or passport number 2 System returns the traveller and flight segment 3 Agent picks a free seat on the cabin map 4 System saves the pass: ticket number and IATA BCBP barcode data 5 Agent tags each bag: 10-digit tag and a check-in scan event
Postconditions	Seat taken, boarding pass issued, bags tagged and scanned

Exception E1 Seat already taken: HTTP 409 Conflict and nothing is saved

UC-03

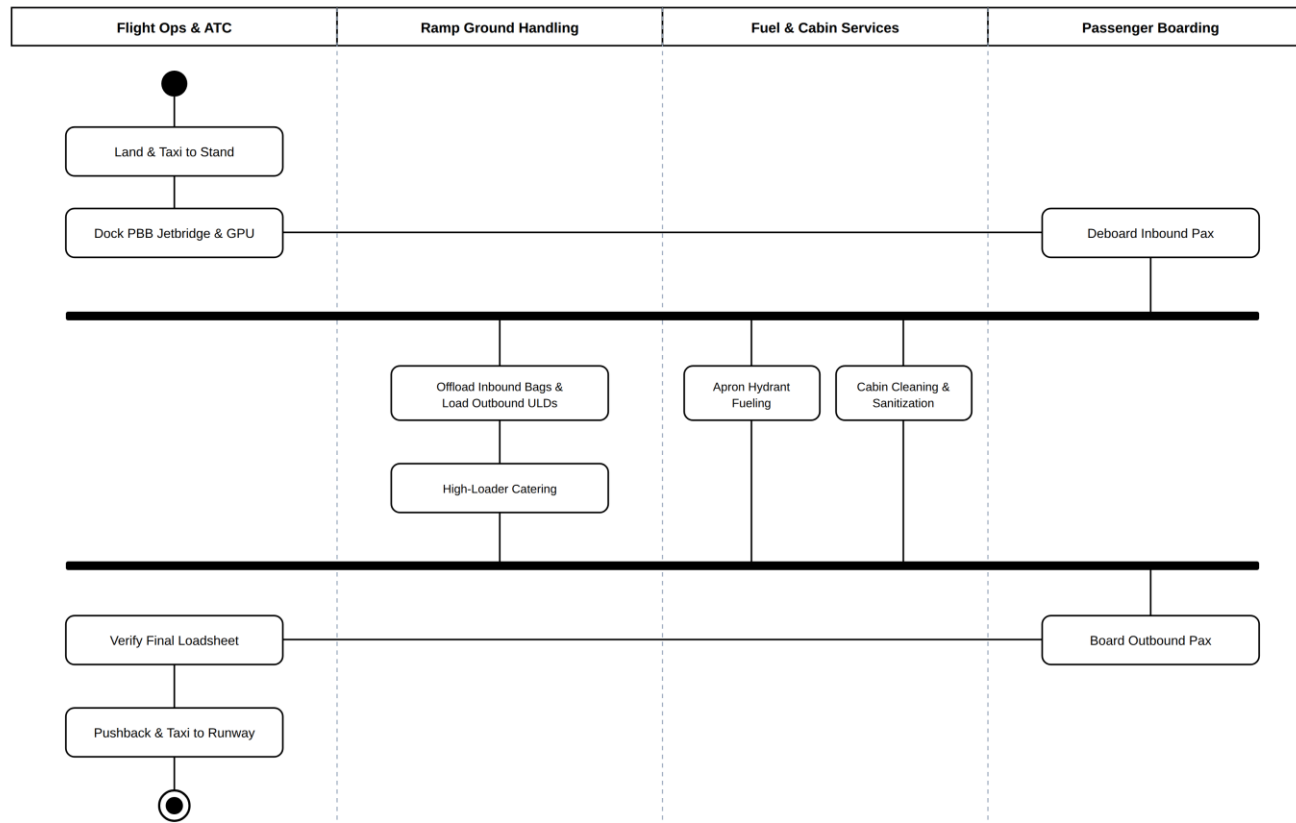
Gate Allocation

Actor	Airside dispatcher or AOCC manager
Preconditions	Flight is scheduled; the target gate is active
Main flow	1 Dispatcher selects a flight and a target gate 2 System locks the gate row, then the flight row 3 Aircraft wingspan is checked against the gate limit 4 Time window is checked against flights at that gate 5 gate_id is saved on the flight
Postconditions	Gate saved on the flight; boards update

Exception E1 Aircraft too wide or slot overlap: HTTP 409 Conflict

Activity Diagram: Aircraft Turnaround

The critical path, with a fork into four parallel servicing streams and a join before boarding



Critical path

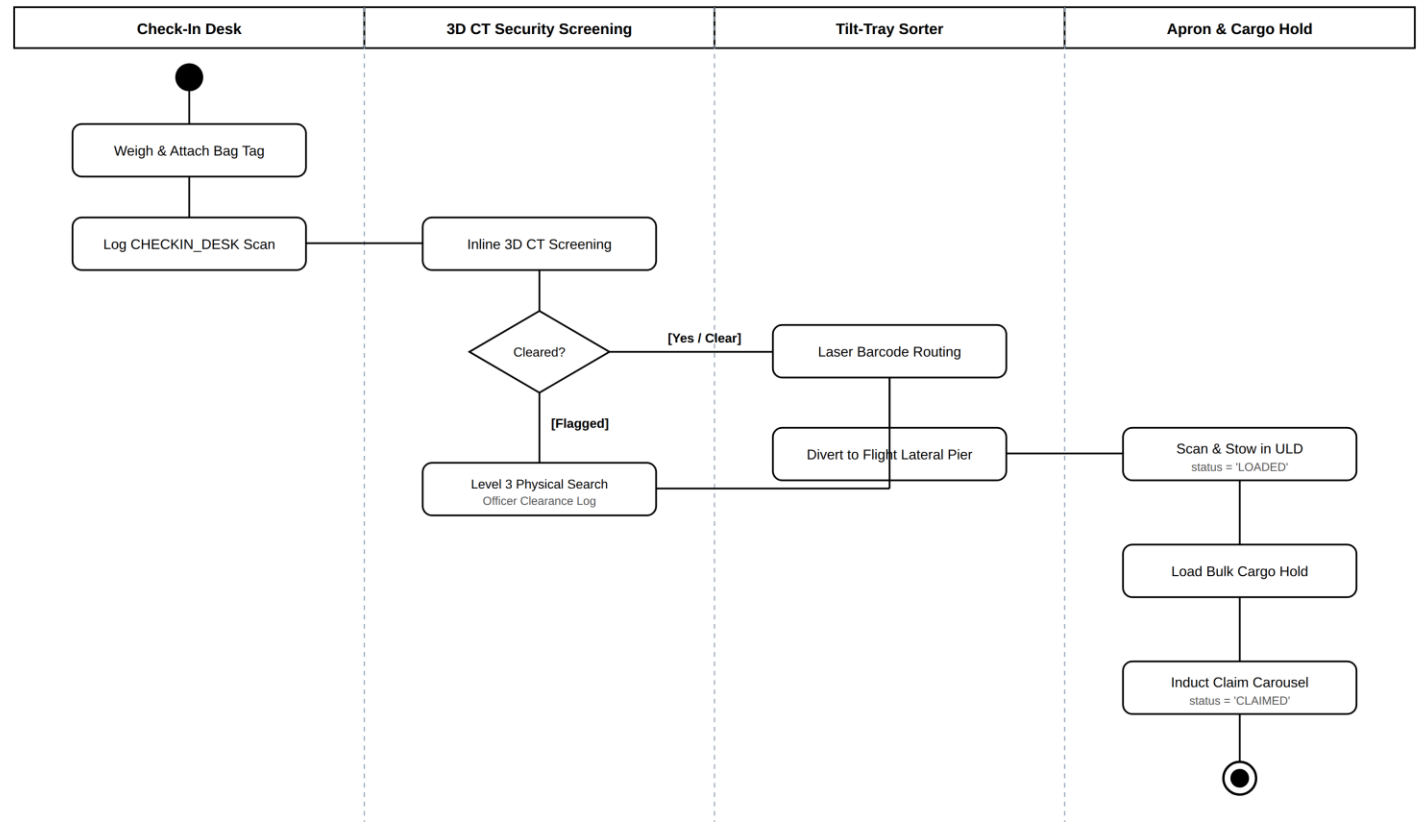
- 1 Arrival**
Land, taxi to stand, dock the jet bridge and ground power, deboard.
- 2 Fork: parallel servicing**
Bags, catering, refuelling and cabin cleaning run at the same time.
- 3 Join: synchronisation**
Boarding starts only after every servicing task is COMPLETED.
- 4 Departure**
Board, verify the final loadsheet, push back and taxi.

Activity Diagram: Baggage Handling

Five scan checkpoints from the check-in desk to the reclaim carousel, with a screening decision

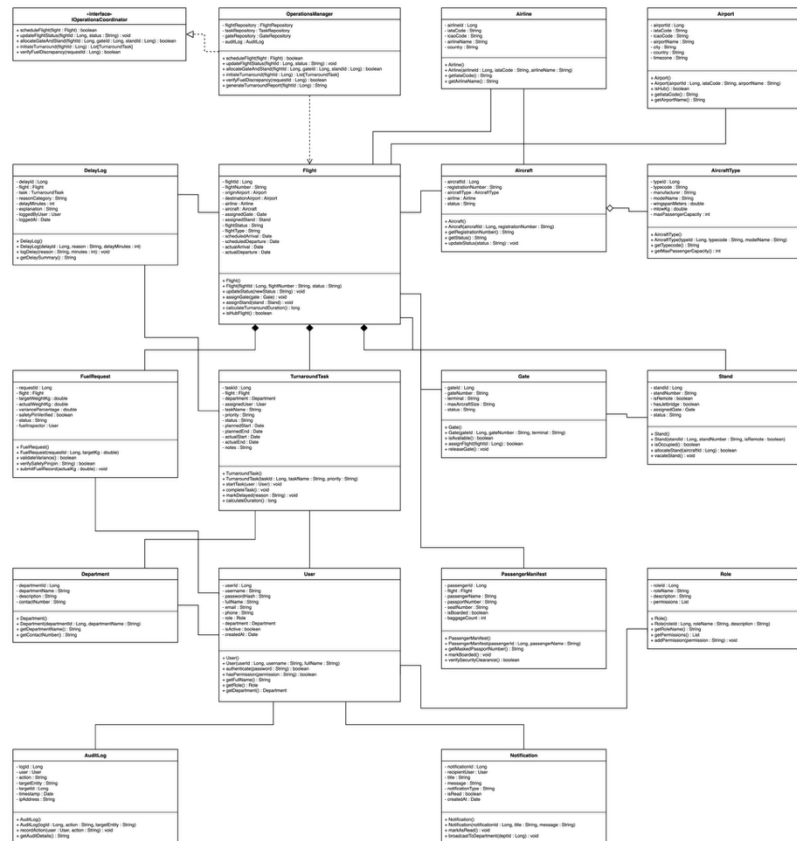
Bag journey

- 1 Check-in**
Bag weighed and tagged; CHECKIN_DESK scan logged.
- 2 3D CT screening**
Cleared bags go on; flagged bags get a Level 3 search.
- 3 Sortation**
Laser barcode routing diverts each bag to its flight lateral.
- 4 Apron loading**
Scanned into the ULD and hold; status LOADED.
- 5 Arrival reclaim**
Inducted onto the claim carousel; status CLAIMED.



UML Class Diagram & Domain Model

Domain classes map directly onto the Spring Data JPA entities, with multiplicities and an interface



Core classes

Flight

Number, status, scheduled and actual times; links airline, aircraft, gate and stand; 1:N turnaround tasks and delay logs.

Aircraft · AircraftType

Type holds wingspan, MTOW and capacity; each aircraft belongs to an airline.

Gate · Stand

Gate has a terminal and maximum aircraft size; stands can be remote and have jet bridges.

TurnaroundTask · User

Tasks are assigned to staff with planned and actual times; users carry a role and a department.

IOperationsCoordinator

Interface realised by OperationsManager: schedule, update status, allocate gate and stand, start turnaround.

Design Patterns in Saphire AOCS

Four patterns that structure the persistence, web and security layers of the backend



Repository

Spring Data JPA interfaces hide SQL behind type-safe query methods, so services never build queries by hand.

FlightRepository

GateRepository

TaskRepository



Global Exception Handler

One `@RestControllerAdvice` turns every exception into a consistent RFC 7807 error body with a correlation ID.

GlobalExceptionHandler

ProblemDetail



Filter Chain

Chain of responsibility: each request passes the logging filter, then the JWT filter that checks the session and loads the role.

RequestLoggingFilter

JwtAuthFilter



Data Transfer Object

DTOs decouple JPA entities from the JSON API and keep fields like password hashes out of responses.

FlightDTO

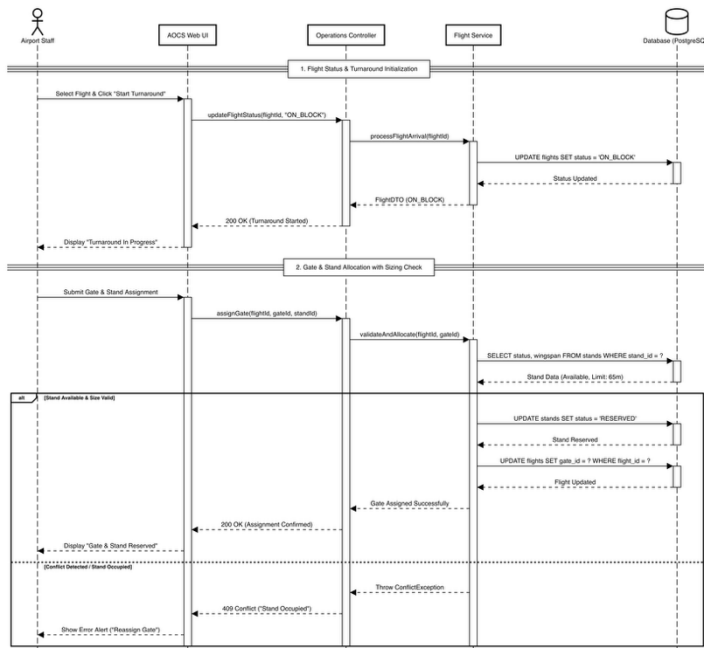
CheckinLookupDTO

AuditLogDTO

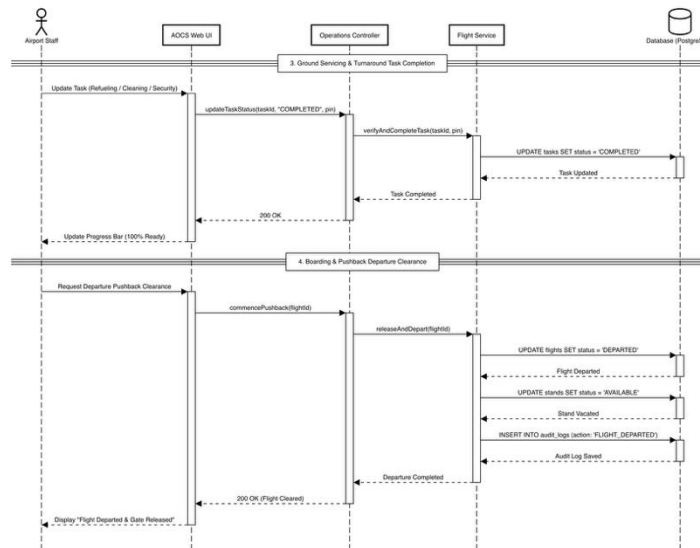
Sequence Diagram: Turnaround & Delays

Lifelines from airport staff through the web UI, controller and service to PostgreSQL

Part 1 · phases 1 and 2



Part 2 · phases 3 and 4

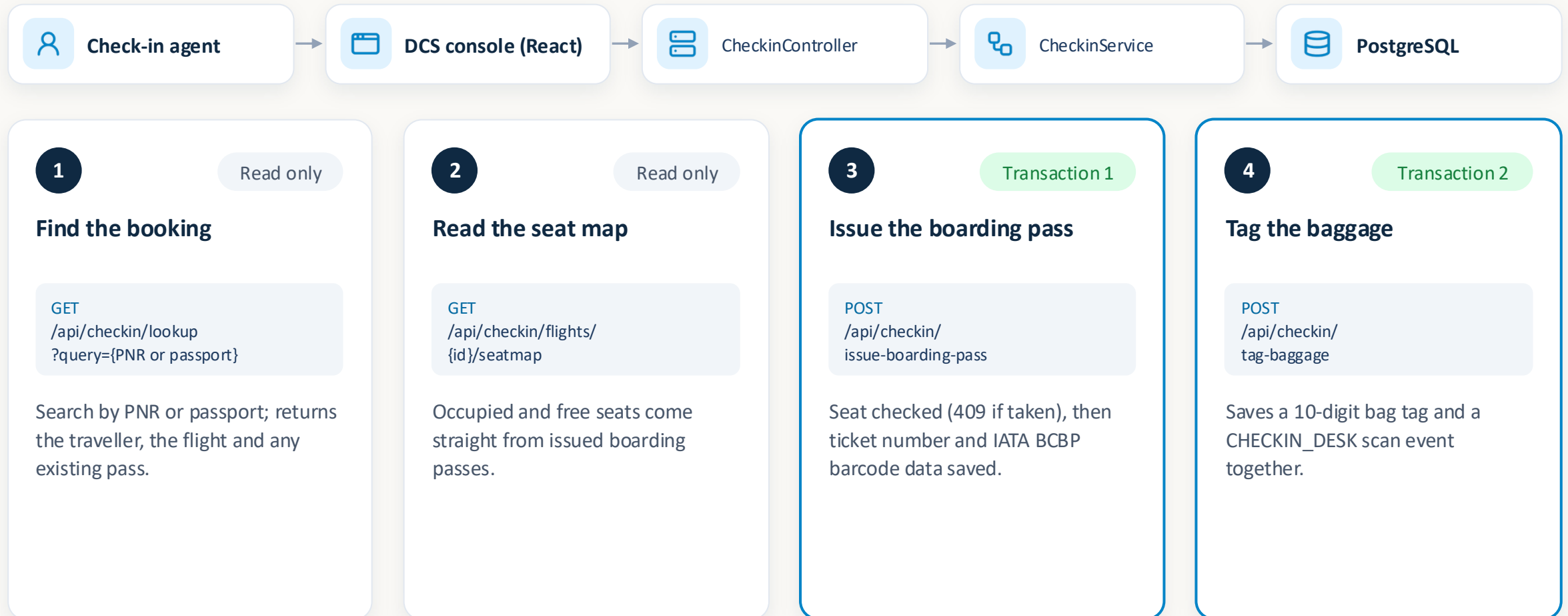


Four phases

- 1 **Turnaround start**
Staff starts the turnaround; the flight is set ON_BLOCK.
- 2 **Gate and stand**
Wingspan and availability checked; a conflict returns 409.
- 3 **Ground servicing**
Tasks set to COMPLETED through PUT /api/tasks/{id}/status.
- 4 **Pushback**
Flight DEPARTED, stand released, audit log written.

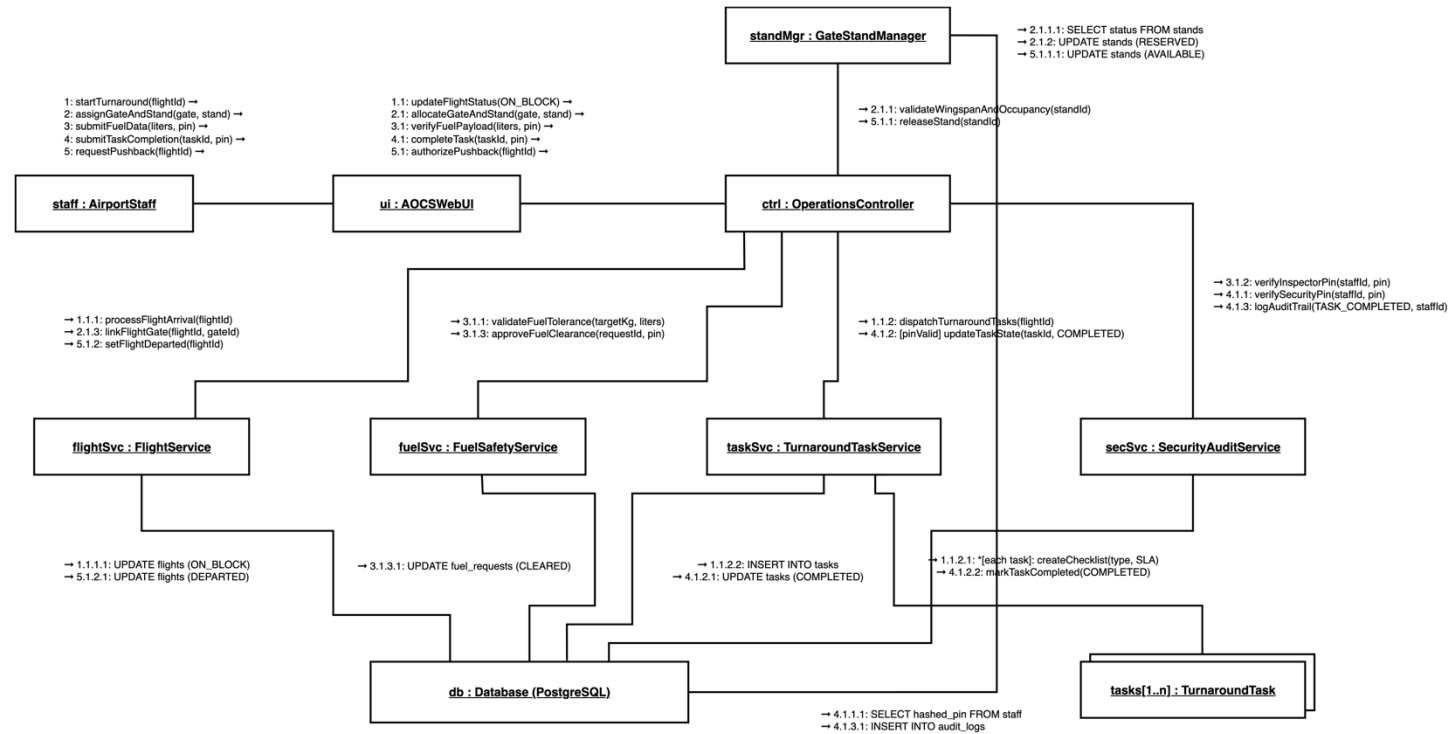
Sequence Flow: Check-In & Boarding Pass

Four calls from the check-in desk to the database, in two separate transactions



Collaboration Diagram: Turnaround Control

Objects linked by numbered messages, from the first ON_BLOCK update to pushback



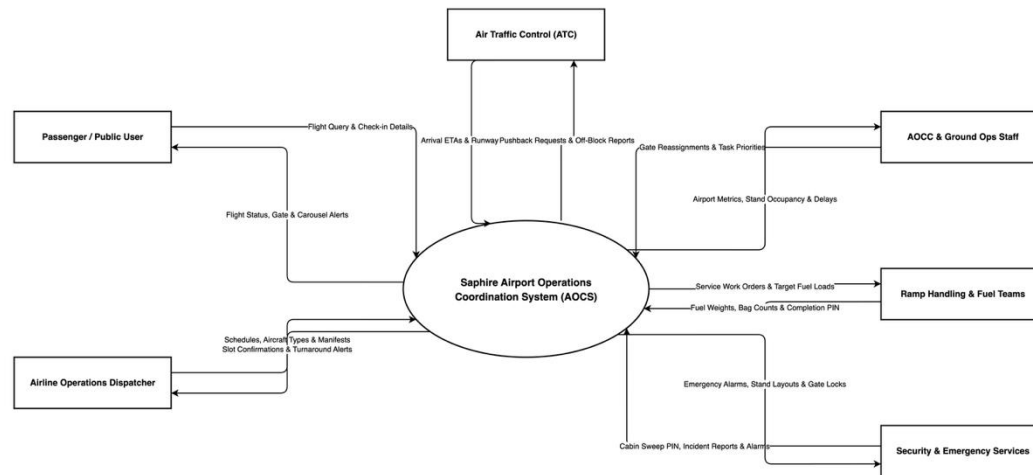
Top-level messages

- 1 startTurnaround(flightId)
Flight set ON_BLOCK; turnaround tasks dispatched.
- 2 assignGateAndStand(gate, stand)
Wingspan and occupancy validated; stand RESERVED.
- 3 submitFuelData(liters, pin)
Fuel tolerance checked; inspector PIN verified.
- 4 submitTaskCompletion(taskId, pin)
Task COMPLETED; audit trail written.
- 5 requestPushback(flightId)
Stand released; flight DEPARTED.

Data Flow Diagrams: Level 0 & Level 1

The system in its context, then decomposed into five processes and five data stores

Level 0 · context diagram



Level 1 · subsystem decomposition



External entities (6)

Airline dispatcher

Passengers

Air traffic control

AOCC & ground staff

Ramp & fuel teams

Security services

Processes (5) · data stores D1 to D5

Flight schedules

Gates & stands

Passengers & bags

Turnaround

Airside security

Relational Database Schema

43 tables in third normal form on PostgreSQL, built by 17 Flyway migrations (V1 to V20)

1. SECURITY & IDENTITY GOVERNANCE (5 TABLES)		2. AIRSIDE & AIRFIELD INFRASTRUCTURE (10 TABLES)		3. AIRSIDE TURNAROUND & GROUND SERVICES (10 TABLES)		4. PASSENGER JOURNEY & BAGGAGE LIFECYCLE (10 TABLES)	
ROLES PCI_ROLE (PK) ROLE (PK) ROLE_DESCRIPTION (FK) ROLE_PERMISSIONS (FK)		AIRLINES PCI_AIRLINE (PK) AIRLINE (PK) AIRLINE_CODE (FK) AIRLINE_DESCRIPTION (FK)		TASKS PCI_TASK (PK) TASK (PK) TASK_CODE (FK) TASK_DESCRIPTION (FK)		TRAVELLERS PCI_TRAVELLER (PK) TRAVELLER (PK) TRAVELLER_CODE (FK) TRAVELLER_DESCRIPTION (FK)	
DEPARTMENTS PCI_DEPARTMENT (PK) DEPARTMENT (PK) DEPARTMENT_CODE (FK) DEPARTMENT_DESCRIPTION (FK)		AIRPORTS PCI_AIRPORT (PK) AIRPORT (PK) AIRPORT_CODE (FK) AIRPORT_DESCRIPTION (FK)		GROUNDS EQUIPMENT PCI_GROUND_EQUIPMENT (PK) GROUND_EQUIPMENT (PK) GROUND_EQUIPMENT_CODE (FK) GROUND_EQUIPMENT_DESCRIPTION (FK)		PASSENGERS PCI_PASSENGER (PK) PASSENGER (PK) PASSENGER_CODE (FK) PASSENGER_DESCRIPTION (FK)	
USERS PCI_USER (PK) USER (PK) USER_CODE (FK) USER_DESCRIPTION (FK)		AIRCRAFT TYPES PCI_AIRCRAFT_TYPE (PK) AIRCRAFT_TYPE (PK) AIRCRAFT_TYPE_CODE (FK) AIRCRAFT_TYPE_DESCRIPTION (FK)		EQUIPMENT ASSIGNMENTS PCI_EQUIPMENT_ASSIGNMENT (PK) EQUIPMENT_ASSIGNMENT (PK) EQUIPMENT_ASSIGNMENT_CODE (FK) EQUIPMENT_ASSIGNMENT_DESCRIPTION (FK)		BOARDING PASSES PCI_BOARDING_PASS (PK) BOARDING_PASS (PK) BOARDING_PASS_CODE (FK) BOARDING_PASS_DESCRIPTION (FK)	
USER_PHONE_NUMBERS PCI_USER_PHONE_NUMBER (PK) USER_PHONE_NUMBER (PK) USER_PHONE_NUMBER_CODE (FK) USER_PHONE_NUMBER_DESCRIPTION (FK)		AIRWAY PCI_AIRWAY (PK) AIRWAY (PK) AIRWAY_CODE (FK) AIRWAY_DESCRIPTION (FK)		FUEL LOGS PCI_FUEL_LOG (PK) FUEL_LOG (PK) FUEL_LOG_CODE (FK) FUEL_LOG_DESCRIPTION (FK)		BAG TAGS PCI_BAG_TAG (PK) BAG_TAG (PK) BAG_TAG_CODE (FK) BAG_TAG_DESCRIPTION (FK)	
AUTH SESSIONS PCI_AUTH_SESSION (PK) AUTH_SESSION (PK) AUTH_SESSION_CODE (FK) AUTH_SESSION_DESCRIPTION (FK)		GATES PCI_GATE (PK) GATE (PK) GATE_CODE (FK) GATE_DESCRIPTION (FK)		SHIFT HANDOVER LOGS PCI_SHIFT_HANDOVER_LOG (PK) SHIFT_HANDOVER_LOG (PK) SHIFT_HANDOVER_LOG_CODE (FK) SHIFT_HANDOVER_LOG_DESCRIPTION (FK)		BAGGAGE SCAN EVENTS PCI_BAGGAGE_SCAN_EVENT (PK) BAGGAGE_SCAN_EVENT (PK) BAGGAGE_SCAN_EVENT_CODE (FK) BAGGAGE_SCAN_EVENT_DESCRIPTION (FK)	
FLIGHT OPERATIONS & HANDOVER (10 TABLES)		CHECKIN COUNTERS PCI_CHECKIN_COUNTER (PK) CHECKIN_COUNTER (PK) CHECKIN_COUNTER_CODE (FK) CHECKIN_COUNTER_DESCRIPTION (FK)		CARGO MANIFESTS PCI_CARGO_MANIFEST (PK) CARGO_MANIFEST (PK) CARGO_MANIFEST_CODE (FK) CARGO_MANIFEST_DESCRIPTION (FK)		MISHANDLED BAGGAGE PCI_MISHANDLED_BAGGAGE (PK) MISHANDLED_BAGGAGE (PK) MISHANDLED_BAGGAGE_CODE (FK) MISHANDLED_BAGGAGE_DESCRIPTION (FK)	
WEATHER REPORTS PCI_WEATHER_REPORT (PK) WEATHER_REPORT (PK) WEATHER_REPORT_CODE (FK) WEATHER_REPORT_DESCRIPTION (FK)		STAGES PCI_STAGE (PK) STAGE (PK) STAGE_CODE (FK) STAGE_DESCRIPTION (FK)		CUSTOMER FEEDBACK LOGS PCI_CUSTOMER_FEEDBACK_LOG (PK) CUSTOMER_FEEDBACK_LOG (PK) CUSTOMER_FEEDBACK_LOG_CODE (FK) CUSTOMER_FEEDBACK_LOG_DESCRIPTION (FK)		SECURITY CHECKPOINTS PCI_SECURITY_CHECKPOINT (PK) SECURITY_CHECKPOINT (PK) SECURITY_CHECKPOINT_CODE (FK) SECURITY_CHECKPOINT_DESCRIPTION (FK)	
GATE ASSIGNMENT RULES PCI_GATE_ASSIGNMENT_RULE (PK) GATE_ASSIGNMENT_RULE (PK) GATE_ASSIGNMENT_RULE_CODE (FK) GATE_ASSIGNMENT_RULE_DESCRIPTION (FK)		FRAMES PCI_FRAME (PK) FRAME (PK) FRAME_CODE (FK) FRAME_DESCRIPTION (FK)		AIRLINE BILLING INVOICES PCI_AIRLINE_BILLING_INVOICE (PK) AIRLINE_BILLING_INVOICE (PK) AIRLINE_BILLING_INVOICE_CODE (FK) AIRLINE_BILLING_INVOICE_DESCRIPTION (FK)		SECURITY INCIDENTS PCI_SECURITY_INCIDENT (PK) SECURITY_INCIDENT (PK) SECURITY_INCIDENT_CODE (FK) SECURITY_INCIDENT_DESCRIPTION (FK)	
FLIGHTS PCI_FLIGHT (PK) FLIGHT (PK) FLIGHT_CODE (FK) FLIGHT_DESCRIPTION (FK)		RUNWAYS PCI_RUNWAY (PK) RUNWAY (PK) RUNWAY_CODE (FK) RUNWAY_DESCRIPTION (FK)		INVOICE LINE ITEMS PCI_INVOICE_LINE_ITEM (PK) INVOICE_LINE_ITEM (PK) INVOICE_LINE_ITEM_CODE (FK) INVOICE_LINE_ITEM_DESCRIPTION (FK)		PASSENGER CLEARANCE LOGS PCI_PASSENGER_CLEARANCE_LOG (PK) PASSENGER_CLEARANCE_LOG (PK) PASSENGER_CLEARANCE_LOG_CODE (FK) PASSENGER_CLEARANCE_LOG_DESCRIPTION (FK)	
RELAY LOGS PCI_RELAY_LOG (PK) RELAY_LOG (PK) RELAY_LOG_CODE (FK) RELAY_LOG_DESCRIPTION (FK)				OPERATIONAL INQUIRIES PCI_OPERATIONAL_INQUIRY (PK) OPERATIONAL_INQUIRY (PK) OPERATIONAL_INQUIRY_CODE (FK) OPERATIONAL_INQUIRY_DESCRIPTION (FK)		IMMIGRATION RECORDS PCI_IMMIGRATION_RECORD (PK) IMMIGRATION_RECORD (PK) IMMIGRATION_RECORD_CODE (FK) IMMIGRATION_RECORD_DESCRIPTION (FK)	
RELAY LOGS PCI_RELAY_LOG (PK) RELAY_LOG (PK) RELAY_LOG_CODE (FK) RELAY_LOG_DESCRIPTION (FK)				LOUNGE VISITS PCI_LOUNGE_VISIT (PK) LOUNGE_VISIT (PK) LOUNGE_VISIT_CODE (FK) LOUNGE_VISIT_DESCRIPTION (FK)		BAGGAGE CAROUSELS PCI_BAGGAGE_CAROUSEL (PK) BAGGAGE_CAROUSEL (PK) BAGGAGE_CAROUSEL_CODE (FK) BAGGAGE_CAROUSEL_DESCRIPTION (FK)	
				LOST AND FOUND ITEMS PCI_LOST_AND_FOUND_ITEM (PK) LOST_AND_FOUND_ITEM (PK) LOST_AND_FOUND_ITEM_CODE (FK) LOST_AND_FOUND_ITEM_DESCRIPTION (FK)		RELAY LOGS PCI_RELAY_LOG (PK) RELAY_LOG (PK) RELAY_LOG_CODE (FK) RELAY_LOG_DESCRIPTION (FK)	

The figure shows 41 tables. The live database adds notifications and security_incidents, for 43 in total.

Six functional domains

Security & identity

5 tables

roles, departments, users, user_phone_numbers, auth_sessions

Aerodrome & airfield

8 tables

airlines, airports, aircraft_types, aircraft, gates, checkin_counters, stands, runways

Flight operations

5 tables

weather_reports, gate_assignment_rules, flights, delay_codes, delay_logs

Airside turnaround

5 tables

tasks, ground_equipment, equipment_assignments, fuel_logs, shift_handover_logs

Passenger & baggage

12 tables

travelers, passengers, boarding_passes, bag_tags, baggage_scan_events, mishandled_baggage, security_checkpoints, passenger_clearance_logs, immigration_records, lounge_visits, baggage_carousels, lost_and_found_items

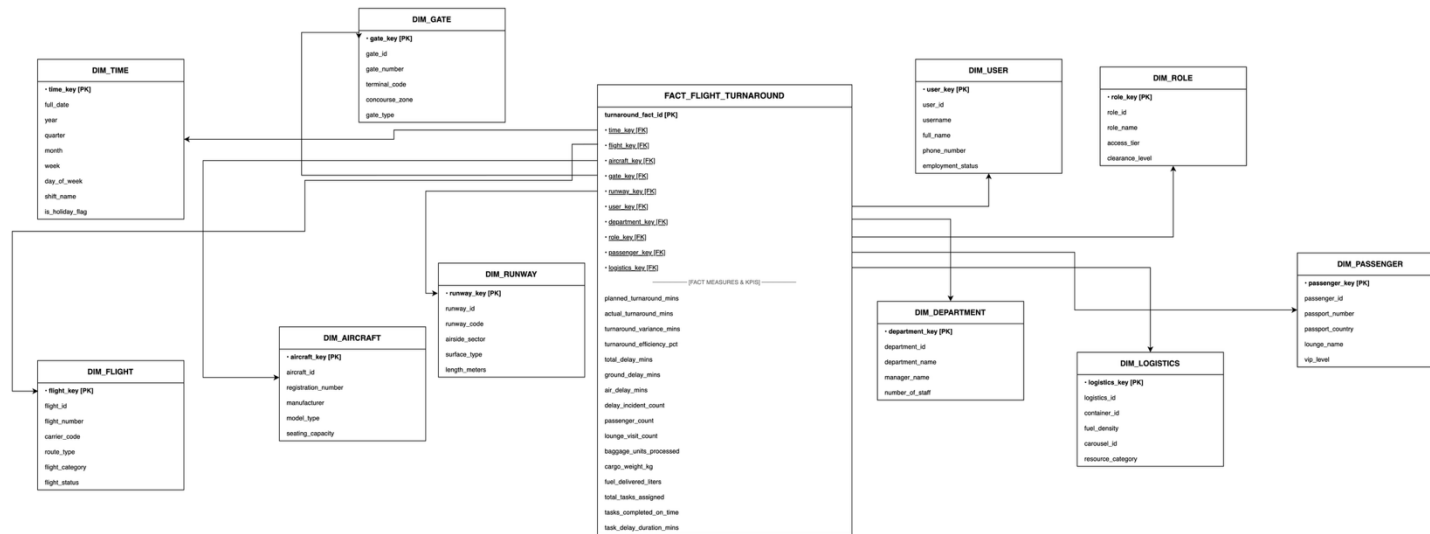
Commercial & audit

6 tables

cargo_manifests, customer_feedback_logs, airline_billing_invoices, invoice_line_items, operational_inquiries, audit_logs

Star Schema for OLAP Analytics

A denormalised model for turnaround KPIs, separate from the operational tables



16 measures in FACT_FLIGHT_TURNAROUND

Planned, actual and variance turnaround minutes · efficiency % · total, ground and air delay minutes · delay incidents · passengers · lounge visits · baggage units · cargo kg · fuel litres · tasks assigned, on time and delayed

OLTP vs OLAP

1	10	16
fact table	dimensions	measures

Grain

One row per completed aircraft turnaround

Why keep it separate

The 43 operational tables stay in 3NF for safe writes. The star schema denormalises them so KPI queries never lock production tables.

Dimensions

Time · flight · aircraft · gate · runway · user · department · role · passenger · logistics

Technology Stack & Integration

Three tiers, one request path: browser to REST API to PostgreSQL, every console on live data



Frontend

Browser · port 3000

- React 19 and TypeScript on Vite 8
- Material UI 9 and Tailwind CSS 4
- Axios client with a JWT interceptor
- React Router 7 with role-guarded routes
- 7 public pages and 9 staff consoles



Backend

REST API · port 8080

- Java 17 and Spring Boot 3.2.5
- Spring Security 6 with stateless JWT
- Role checks with @PreAuthorize
- Spring Data JPA and Hibernate
- 22 REST controllers and DTOs

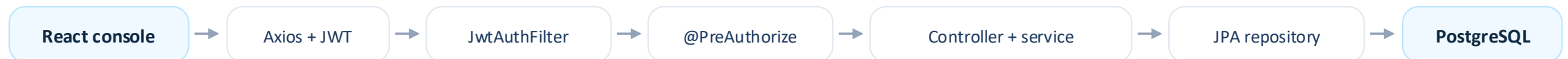


Data & Infrastructure

PostgreSQL · port 5432

- PostgreSQL 16 or newer
- 17 Flyway migrations, V1 to V20
- auth_sessions: one active session per user
- Docker Compose with an Nginx proxy
- CI boots the app on Postgres 16

Request path for one console action



Locally the Vite dev server proxies /api to port 8080; in Docker, Nginx does the same.

Testing & Quality Assurance

109 automated JUnit 5 and Mockito tests, run in CI on every push



Equivalence Partitioning

Valid and invalid flight status transitions:
SCHEDULED to BOARDING passes, SCHEDULED to LANDED is rejected.



Boundary Values

Login logout: one failure below the limit stays open, the failure at the limit locks the account.



State Transitions

Turnaround tasks and flights may only move along their state machines; illegal jumps are refused.



Regression in CI

Every push runs the suite, then boots the app on an empty PostgreSQL 16 to prove migrations run.

```
./mvnw test
```

```
✓ TurnaroundTaskServiceTest 17
✓ UserServiceTest 10
✓ GateServiceTest 9
✓ LoginAttemptServiceTest 9
✓ FlightControllerValidationTest 9
✓ FlightServiceTest 14
✓ AuthServiceTest 7
✓ SecurityAuditServiceTest 6
✓ SecurityOpsServiceTest 6
✓ 8 more test classes 22
```

Tests run: 109

Failures: 0 Errors: 0

Live Consoles & Public Portal

16 pages on live data: seven public pages and nine role consoles behind server-checked sessions

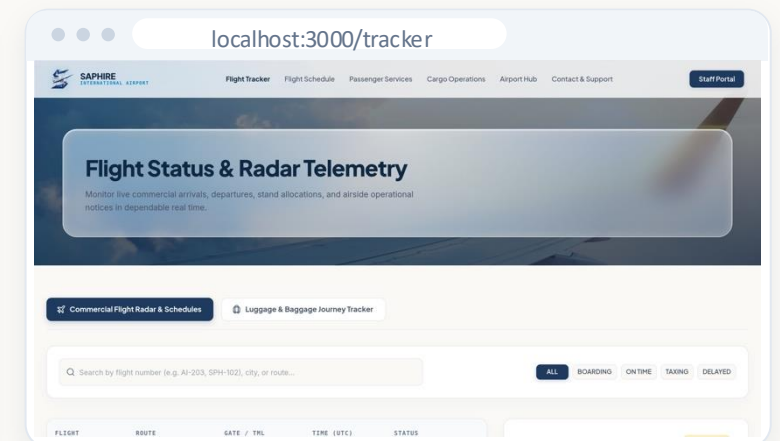
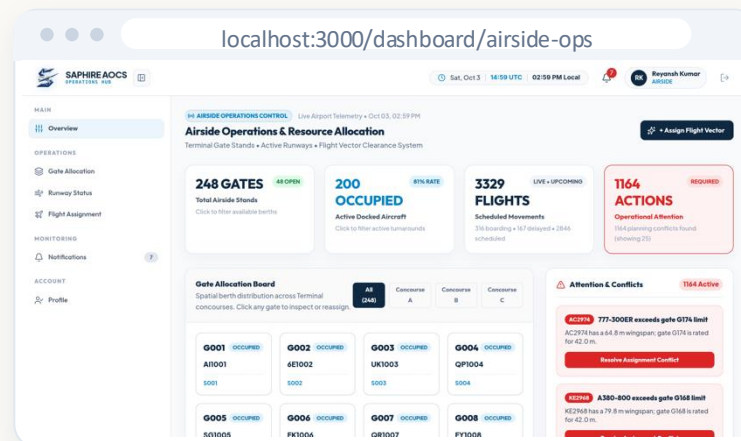
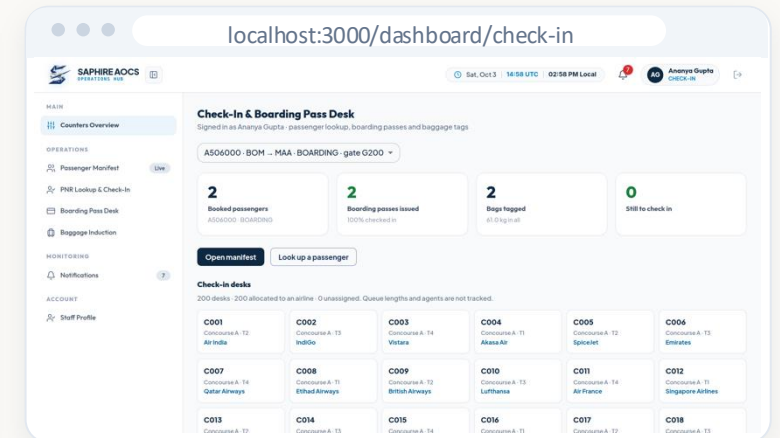
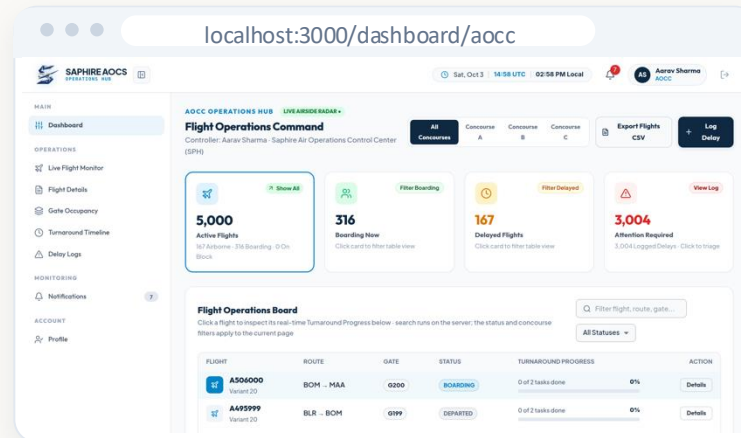
Public portal

Home /
Flight tracker /tracker
Timetable /schedule
Passenger services /passenger-services
Cargo /cargo
Airport guide /airport
Support /contact

Staff consoles

under /dashboard/

Check-in desk check-in
AOCC controller aocc
Ground operations ground-ops
Airside operations airside-ops
Logistics & BHS logistics
Passenger security passenger-security
Airline billing billing
System admin system-admin
Department services department



Conclusion & Future Roadmap

What was built, what is distinctive, and where the platform goes next



Engineering Summary

- Full-stack airport operations platform across nine lab experiments
- 43 tables, about 158,000 seed records, 109 automated tests
- 16 pages wired to a live PostgreSQL database



Key Innovations

- Wingspan vs gate conflict detection (HTTP 409)
- Server-side session revocation through `auth_sessions`
- Shift handover logbook with digital sign-off
- CSV report exports for flights and billing



Future Roadmap

- Machine-learning turnaround delay forecasting
- RFID and IoT apron vehicle tracking over MQTT
- DigiYatra biometric e-gates integration

Krishna Solanki · Chaitanya Tikku · Anuvrat Tripathi · Anay Modi